

Block-by-Block Mapping of the Bird Deterrence System

1 Block-by-Block System Mapping

The implemented closed-loop system consists of local intensity modeling, hotspot and candidate generation, selective preventive admission, dispatch, execution, and intervention feedback. The following subsections map each diagram block to its description, mathematical formulation, and implementation location in the codebase.

1.1 1. Environment and Inputs

Description. This block defines the vineyard geometry, static value weighting, robot profiles, and incoming bird detections. The production simulator uses row structure and edge exposure to build a value map. Robot profiles encode speed, endurance, battery, health, and deterrence capability.

Mathematics. The planning field is conceptually defined as

$$q(x, t) = w(x, t)\lambda(x, t). \quad (1)$$

In the current implementation, the static value map is effectively

$$w(x, y) = 1 + w_{\text{row}}(y) + w_{\text{edge}}(x, y), \quad (2)$$

where

$$w_{\text{row}}(y) = \begin{cases} \text{row_gain}, & \text{if } \text{dist_to_row_center}(y) \leq \frac{\text{row_width}}{2}, \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

and

$$w_{\text{edge}}(x, y) = \text{edge_gain} \exp\left(-\frac{d_{\text{edge}}(x, y)}{\text{edge_scale}}\right). \quad (4)$$

A coarser fallback weighting used during counterfactual scoring is

$$w(x, y) = 1 + w_{\text{edge}}(x, y) + w_{\text{block}}(x, y), \quad (5)$$

where w_{block} is a sinusoidal spatial modulation term.

Implementation.

- `DeterrentSystem.py`: `value_weight(x, y)`
- `DeterrentSystem.py`: robot profile setup inside `run_simulation_frames_persistent(...)`
- `Robot.py`: `RobotProfile`
- `TaskGenerator.py`: `_weight_at(...)` and `estimate_counterfactual_reduction(...)`

1.2 2. Health-Weighted Zone Partitioning

Description. This block assigns robots to spatial responsibility regions using a power-diagram or Laguerre-style partition based on anchor positions and health-derived weights. It also extracts a neighbor graph for local coordination.

Mathematics. Conceptually, zone ownership is given by

$$r^*(x) = \arg \min_r (\|x - a_r\|^2 - \pi_r), \quad (6)$$

where a_r is the anchor of robot r and π_r is a health-derived power weight.

The weight is computed through

$$\pi_r = \text{health_to_weight}(h_r; \text{mode}, \text{scale}, \gamma), \quad (7)$$

followed by cell generation using

$$\text{power_cells}(\text{anchors}, \text{weights}, \text{boundary}). \quad (8)$$

Implementation.

- `DeterrentSystem.py`: `build_fleet_and_zones(...)`
- `DeterrentSystem.py`: calls to `ZonePartitioner(...)`, `update_anchor(...)`, `recompute(force=True)`, `cells_for_ids()`, and `neighbors_for_id(...)`
- Internal logic resides in `ZonePartitioner.py` (not uploaded)

1.3 3. Detection Attribution and Local / Boundary Event Ingestion

Description. Each detection is attributed to the robot that owns the corresponding zone. If the detection lies inside the robot's zone, it updates that robot's local model and recent-event history. If it lies near a zone boundary, it is also shared with neighboring robots with reduced influence.

Mathematics. Let E_r denote local events for robot r . Then:

- if $x_i \in \Omega_r$, add (x_i, t_i) to E_r ,
- if $d(x_i, \partial\Omega_r) \leq d_b$, also share the event with neighbors $k \in N(r)$.

Thus the event set used by robot r is

$$E_r^{\text{used}} = E_r \cup E_{N(r)}^{\text{shared}}. \quad (9)$$

Implementation.

- `Robot.py`: `ingest_detection(...)`
- `Robot.py`: `ingest_boundary_event(...)`
- `DeterrentSystem.py`: `EventBus.send_boundary_events(...)`

1.4 4. Per-Robot Online SESTPP Model

Description. Each robot maintains a grid-based online spatiotemporal point-process model with local self-excitation, cross-excitation from neighboring boundary events, and mode-specific inhibition from completed deterrence actions.

Mathematics. The baseline model is

$$\lambda_r(x, t) = \mu_r(x)s(t) + \sum_{(x_i, t_i) \in E_r} \alpha_{\text{in}} K_\sigma(x - x_i) e^{-(t-t_i)/\omega} + \sum_{(x_k, t_k) \in E_{N(r)}} \alpha_{\text{cross}} w_{r \leftarrow k} K_\sigma(x - x_k) e^{-(t-t_k)/\omega}. \quad (10)$$

The intervention-aware extension is

$$\lambda_r^{IA}(x, t) = \left[\lambda_r(x, t) - \sum_{(z_j, \tau_j, m_j) \in U_r} \beta(m_j, x, \tau_j) K_{\sigma_u(m_j)}(x - z_j) e^{-(t-\tau_j)/\omega_u(m_j)} \right]_+. \quad (11)$$

In the implementation, the update is summarized as

$$\text{lam} = \text{clip}(\mu \cdot \text{time_multiplier} + \text{trigger_mass} - \text{inhib_mass}, 0, \infty). \quad (12)$$

Decay is exponential:

$$\text{trigger_mass}(t + \Delta t) = \text{trigger_mass}(t) e^{-\Delta t/\omega}, \quad (13)$$

and each inhibition channel decays as

$$\text{inhib_channel}(t + \Delta t) = \text{inhib_channel}(t) e^{-\Delta t/\omega_u}. \quad (14)$$

Implementation.

- SESTPP.py: `OnlineSESTPP.__init__(...)`
- SESTPP.py: `advance_time(dt)`
- SESTPP.py: `add_local_event(...)`
- SESTPP.py: `add_cross_event(...)`
- SESTPP.py: `add_intervention_event(...)`
- SESTPP.py: `_update_lambda()`
- DeterrentSystem.py: one `OnlineSESTPP` object per robot

1.5 5. Predicted Intensity Field, Value-Weighted Demand, and Hotspot Extraction

Description. This block turns the predicted intensity into a planning field. In the current implementation, hotspots are extracted from the *excess field* $\lambda - \mu$, while value weighting is applied later during task scoring.

Mathematics. Conceptually,

$$q(x, t) = w(x, t)\lambda(x, t). \quad (15)$$

In the current code, hotspot scoring is based on

$$\text{field_score}(x, y) = \lambda(x, y) - \mu(x, y). \quad (16)$$

Patrol benefit is computed as

$$\text{benefit}_{\text{patrol}}(x, y) = \max(0, \lambda - \mu) w(x, y) \omega \left(1 - e^{-T_H/\omega}\right) \Delta A. \quad (17)$$

The patrol utility is then

$$\text{utility}_{\text{patrol}} = \text{benefit}_{\text{patrol}} - \text{cost}_{\eta}. \quad (18)$$

Implementation.

- SESTPP.py: `hotspots(top_k, merge_radius, use_excess=True, mask_poly=...)`
- TaskGenerator.py: `_field_score(x,y)`
- TaskGenerator.py: `_patrol_hotspots(...)`
- TaskGenerator.py: `_patrol_candidate_metrics(...)`

1.6 6. Preventive Candidate Generation and Expected Future Reduction

Description. Preventive deterring candidates are created from recent detection clusters, model hotspots, and border hotspots near zone boundaries. Each candidate is then evaluated using a counterfactual expected-future-reduction score.

Mathematics. The thesis-level preventive objective is

$$\Delta J(a) = \int_t^{t+T_H} \int_{\Omega_r} w(x) (\lambda_r^{IA}(x, \tau) - \lambda_r^{IA,a}(x, \tau)) dx d\tau - c_E(a). \quad (19)$$

The implementation uses the exponential-kernel shortcut

$$\int_t^{t+T_H} e^{-(\tau-t)/\omega_u} d\tau = \omega_u \left(1 - e^{-T_H/\omega_u}\right). \quad (20)$$

The approximation used in code is

$$\text{available_integral}(x) = \max(\lambda - \lambda_{\text{baseline}}, 0) \cdot \omega \left(1 - e^{-T_H/\omega}\right), \quad (21)$$

$$\text{suppression_here}(x) = \alpha_{\text{inhib}} \beta_u K_{\sigma_u}(x - z) \cdot \omega_u \left(1 - e^{-T_H/\omega_u}\right), \quad (22)$$

$$\text{reduction_here} = \min(\text{available_here}, \text{suppression_here}), \quad (23)$$

$$\text{predicted_deltaJ} = \sum_x w(x) \text{reduction_here}(x) \Delta A, \quad (24)$$

$$\text{utility} = \text{predicted_deltaJ} - \text{cost}_{\eta}, \quad (25)$$

$$\frac{\Delta J}{\text{cost}} = \frac{\text{predicted_deltaJ}}{\text{cost}_{\eta}}. \quad (26)$$

The event-probability proxy is

$$p_{\text{event}} = 1 - e^{-\lambda_{\text{patch}}}. \quad (27)$$

Implementation.

- `TaskGenerator.py: cluster_points(...)`
- `TaskGenerator.py: _collect_border_hotspots(...)`
- `TaskGenerator.py: estimate_counterfactual_reduction(...)`
- `TaskGenerator.py: _deterrence_candidate_metrics(...)`
- `TaskGenerator.py: _event_probability(...)`

1.7 7. Selective Preventive Admission: SPRT, Persistence, Chance, Utility Ratio, Capacity

Description. This block determines whether a preventive candidate should be admitted. In the thesis-facing runtime preset, preventive admission is explicitly governed by the `sprt_capacity` policy.

Mathematics. For a detection-cluster candidate, the sequential log-likelihood ratio is

$$\text{LLR}_k = \text{LLR}_{k-1} + n_k \log\left(\frac{\mu_1}{\mu_0}\right) - (\mu_1 - \mu_0), \quad (28)$$

where n_k is the recent detection count, μ_0 is the baseline expected count, and μ_1 is the alternative count estimate.

The SPRT thresholds are

$$A = \log\left(\frac{1 - \beta}{\alpha}\right), \quad B = \log\left(\frac{\beta}{1 - \alpha}\right). \quad (29)$$

Admission requires

$$\text{LLR} \geq A + \text{margin}, \quad (30)$$

and also

$$p_{\text{event}} \geq p_{\min}, \quad \Delta J \geq \Delta J_{\min}, \quad \frac{\Delta J}{\text{cost}} \geq \rho_{\min}. \quad (31)$$

For accepted detection-cluster candidates, the selection weight is

$$\text{selection_weight} = \text{posterior}_{H_1} \cdot \max(\text{margin_conf}, \text{excess_conf}), \quad (32)$$

and the final preventive score is

$$\text{score}_{\text{preventive}} = \text{selection_weight} \cdot \Delta J - \text{cost}_{\eta}. \quad (33)$$

Implementation.

- `TaskGenerator.py`: SPRT threshold calculations and LLR update
- `TaskGenerator.py`: chance, predicted- ΔJ , utility-ratio, persistence, and capacity filters
- `TaskGenerator.py: _detering_selection_weight(...)`
- `planner_profiles.py`: `THESIS_CALIBRATED_SELECTIVE_PROPOSED_PROFILE_ID`

1.8 8. Dispatch Ordering

Description. After candidate tasks are created, the system orders them before assignment. Direct-detection deterring tasks form an absolute-priority tier. Patrol and model-scored preventive tasks are then ordered lexicographically.

Mathematics. The regular competition tier is sorted by

$$(\text{utility}, \text{predicted_deltaJ}, \text{deltaJ_per_cost}, -\eta_s, p_{\text{event}}, \text{selection_weight}, \text{support}, -t), \quad (34)$$

so higher utility, larger predicted reduction, better utility-per-cost, and smaller ETA rank earlier.

Implementation.

- DeterrentSystem.py: `_dispatch_priority_components(...)`
- DeterrentSystem.py: `_dispatch_priority_sort_key(...)`
- DeterrentSystem.py: `_dispatch_ordering_bucket(...)`
- DeterrentSystem.py: `_ordered_dispatch_candidates(...)`

1.9 9. Task Assignment and Queue Admission

Description. Tasks are assigned to robots using capability, ETA, endurance, zone ownership, health, priority, task value, and current load. Detering tasks are ranked globally because they are time-critical, while patrol is local-first with neighbor fallback.

Mathematics. The assignment score is

$$A(r, \tau) = w_{\text{cap}} \text{cap}_r - w_{\eta} \eta(r, \tau) + w_{\text{stay}} \text{endurance}_r + w_{\text{zone}} \mathbf{1}[x_{\tau} \in \Omega_r] + w_{\text{health}} h_r + w_{\text{prio}} p_{\tau} + w_{\text{task_value}} v_{\tau} - w_{\text{load}} L_r. \quad (35)$$

The ETA is

$$\eta(r, \tau) = \frac{\|x_{\tau} - p_r\|}{v_r} + \text{spinup}(r). \quad (36)$$

Implementation.

- TaskGenerator.py: `TaskAssigner.assign_task(...)`
- TaskGenerator.py: `TaskAssigner._score(...)`
- TaskGenerator.py: `_eligible(...)`
- DeterrentSystem.py: construction of `TaskAssigner(...)`

1.10 10. Motion Execution

Description. Assigned robots move toward their goals under lane and headland constraints. In the simulator, movement is implemented as local first-order stepping. Travel distance and energy are accumulated at each step.

Mathematics. Let the current pose be p_k , the goal be g_k , speed be v_r , and time step be Δt . Then

$$d_k = \|g_k - p_k\|, \quad s_k = v_r \Delta t. \quad (37)$$

The position update is

$$p_{k+1} = \begin{cases} g_k, & s_k \geq d_k, \\ p_k + \frac{s_k}{d_k}(g_k - p_k), & s_k < d_k. \end{cases} \quad (38)$$

Energy is updated as

$$E_r \leftarrow E_r + e_{\text{per m},r} \cdot \text{step_dist}. \quad (39)$$

Arrival occurs when

$$\|p - g\| \leq \text{arrival_radius}. \quad (40)$$

Implementation.

- `DeterrentSystem.py: _project_to_lane(...)`
- `DeterrentSystem.py: _effective_goal(...)`
- `DeterrentSystem.py: _step_to(...)`
- `DeterrentSystem.py: _match_active_task_for_robot_goal(...)`

1.11 11. Deterrence Action Completion

Description. When a robot reaches a deterring goal, it holds position for a fixed dwell time, then the task is marked complete. Completion generates the intervention feedback that will be written back into the model.

Mathematics. Completion occurs when

$$\|p_r - g_r\| \leq r_{\text{arrival}}, \quad (41)$$

followed by a hold condition

$$t \geq t_{\text{start_hold}} + T_{\text{hold}}. \quad (42)$$

Once complete, the system emits intervention feedback parameters

$$(x, y, t, m, \beta, \sigma_u, \omega_u). \quad (43)$$

Implementation.

- `DeterrentSystem.py: _start_deterring_hold(...)`
- `DeterrentSystem.py: _complete_deterring_task(...)`

1.12 12. Local Intervention Feedback

Description. A completed deterrence action is written back into the same robot's model as an inhibition event. This closes the local control loop by allowing completed actions to change future predicted risk.

Mathematics. At completion time $t = \tau_j$, insert intervention

$$(z_j, \tau_j, m_j) \in U_r \quad (44)$$

and add inhibition

$$\alpha_{\text{inhib}} \beta_u K_{\sigma_u} (x - z_j) e^{-(t - \tau_j)/\omega_u} \quad (45)$$

to the selected mode channel.

Implementation.

- `DeterrentSystem.py`: `_complete_detering_task(...)` calls `robots[rid].ingest_intervention_event(...)`
- `Robot.py`: `ingest_intervention_event(...)`
- `SESTPP.py`: `add_intervention_event(...)`

1.13 13. Neighbor Intervention Sharing via EventBus

Description. If a completed deterrence action occurs near a zone boundary, it is also broadcast to neighboring robots. The EventBus applies debounce and minimum-weight filters to reduce communication load. Neighbors ingest the event as cross-inhibition.

Mathematics. If

$$d(z_j, \partial\Omega_r) \leq d_b, \quad (46)$$

broadcast

$$(z_j, \tau_j, m_j, \beta_j, \sigma_{u,j}, \omega_{u,j}) \quad (47)$$

to each neighbor $k \in N(r)$.

Message filtering is governed by:

- send only if weight $\geq w_{\min}$,
- suppress repeated sends within `min_interval_s` after spatial quantization.

Implementation.

- `Robot.py`: `intervention_boundary_events(...)`
- `DeterrentSystem.py`: `EventBus.send_intervention_events(...)`

1.14 14. Telemetry and Metrics

Description. This block measures system performance. The main thesis metric is value-weighted exposure. Additional runtime metrics include response time, travel distance, energy use, completed tasks, task efficiency, and communication overhead.

Mathematics. The main exposure metric is

$$J_{\text{exp}} = \int_0^T \int_{\Omega} w(x) \lambda_{\text{true}}(x, t) dx dt. \quad (48)$$

Communication overhead is approximated as

$$\text{bytes} = (\text{boundary_count}) \cdot b_{\text{boundary}} + (\text{intervention_count}) \cdot b_{\text{intervention}}. \quad (49)$$

Additional metrics include mean response time, robot travel distance, energy use, and completed tasks per distance traveled.

Implementation.

- `DeterrentSystem.py`: runtime accumulators such as `response_times`, `travel_distance_by_robot`, `energy_by_robot`, `completed_count_by_type`, and `value_weighted_exposure`
- `DeterrentSystem.py`: `EventBus` message counters and byte totals
- `SYSTEM_VARIABLES.md`: communication and metric parameter definitions

2 Summary

The implemented closed-loop system can be summarized as follows:

1. SESTPP maintains a per-robot predictive risk field.
2. Task generation converts that field into patrol and preventive candidates.
3. Selective gating decides whether preventive action is worth limited robot capacity.
4. Dispatch and assignment select which robot executes which task.
5. Execution completes deterrence actions and feeds intervention effects back into the model locally and across boundaries.